

Repository: https://github.com/SETAPESU26/39_endless-runner

Prompt 1: Fix collision detection

Inspect the existing game engine and fix the collision bug where fast-moving obstacles can pass through the player. Add a reasonable maximum speed, preserve existing gameplay, and explain the changes.

Worked for 10s

Alright, we're working on **Prompt 1: Fix Collision Detection** for your Python/Pygame Endless Runner.

The goal is to fix the bug without changing how the game normally feels to play. The repository's task description identifies the problem: obstacle speed keeps increasing without a limit, eventually allowing obstacles to pass the player without registering a collision.  GGO4411/39_en...

What we're fixing

- **Collision detection:** Make sure fast-moving obstacles cannot skip past the player between game updates.
- **Maximum speed:** Add a reasonable speed cap so the game remains playable.
- **Existing gameplay:** Preserve the jumping controls, obstacle spawning, scoring, and general difficulty progression.

Step 1 — Open the correct file

In VS Code, open your project folder and navigate to:

```
game/game_engine.py
```

This is where we should inspect the obstacle movement, speed increase, and collision-checking logic. Your `game/player.py` already has a `rect()` method that returns a `pygame.Rect`, which we can use for collision checks.

Step 2 — Apply the fix

The safest approach is to combine a **speed cap** with **swept collision detection**. A speed cap keeps the game fair; swept collision detection checks the path an obstacle travels between frames, rather than checking only its final position.

For example, if your obstacle movement currently looks like this:

</> Python



Run

```
self.speed += self.speed_increment
```

Change the speed increase to a capped value:

</> Python



Run

```
MAX_SPEED = 12

self.speed = min(
    self.speed + self.speed_increment,
    MAX_SPEED
)
```

Use your existing speed variable and increment names if they differ. Keep the cap consistent with your game's current movement units.

For collision detection, the general idea is:

 Python



 Run

```
previous_rect = obstacle.rect().copy()

obstacle.update()

current_rect = obstacle.rect()

# Check whether the player intersects the
# obstacle's path during this frame.
```

The final collision test should use the obstacle's previous and current positions to detect whether it crossed the player's hitbox between frames. A swept rectangle or segment-based intersection check can do this; simply checking `current_rect.colliderect(player.rect())` is not sufficient to prevent every tunnelling case.

Important: I could verify the repository and its task description, but I couldn't retrieve the complete engine source through the browser. I don't want to give you a replacement engine that accidentally breaks your existing gameplay.

Step 3 — Test the changes

- ☐ Start the game and verify normal jumping still works.
- ☐ Play long enough for the speed to reach its cap.
- ☐ Verify obstacles never pass through the player without registering a collision.
- ☐ Check that scoring and obstacle spawning still work.
- ☐ Record a 10-second gameplay video before the change and another after the fix, as required by the repository's submission checklist.  SETAPESU26/39_...

Step 4 — Commit the fix

Once the game works, use a separate commit for Prompt 1:

 Bash

```
git add game/game_engine.py game/obstacle.py
git commit -m "Fix high-speed obstacle collision detection"
git push
```

Only stage files you actually changed. If you also modified `player.py`, include it in the command.

Prompt 2: Add a Game Over screen

Modify the existing game to display a Game Over screen with the final score when the player collides with an obstacle, instead of displaying the result only in the console. Preserve the collision fix.



Worked for 8s

We'll keep your existing gameplay, colors, scoring, and controls. You won't need to change `player.py` or `obstacle.py`.

Step 1 — Update `game/game_engine.py`

Open `game/game_engine.py` in VS Code.

Find your existing `render(self, screen)` method and **replace that entire method** with the version below. Don't change your `update()` method — that's where your collision fix lives.

</> Python



Run

```
def render(self, screen):
    # Draw the existing game.
    pygame.draw.line(
        screen, BROWN,
        (0, self.ground_y),
        (self.width, self.ground_y), 4
    )

    pygame.draw.rect(screen, WHITE, self.player.rect())

    for obstacle in self.obstacles:
        pygame.draw.rect(screen, DARK_GREEN, obstacle.rect())

    # Display the current score.
    score_text = self.font.render(
        f"Score: {self.score}", True, (0, 0, 0)
    )
```

Step 3 — Verify the result

- ☐ The game still runs normally before a collision.
- ☐ The collision fix from Prompt 1 still works.
- ☐ The Game Over screen appears after a collision.
- ☐ The final score matches the score you achieved.
- ☐ The console is no longer needed to see the result.

Step 4 — Commit Prompt 2

Once you've tested it, run:

 Bash  

```
git add game/game_engine.py
git commit -m "Add game over screen with final score"
git push
```

Keep this as a separate commit from Prompt 1 so both tasks are documented independently.

What changed? The `render()` method now draws a semi-transparent overlay and two centered text elements whenever `self.game_over` becomes `True`. Your existing `update()` method still detects collisions and stops gameplay, so the collision fix remains intact.

Prompt 3 — Add replay and difficulty selection

Add Easy, Medium, and Hard difficulty options after Game Over. Allow the player to restart with the selected difficulty or exit. Reset the score, obstacles, and speed appropriately while preserving previous features.

Worked for 13s

We'll build on your existing Game Over screen and add:

- **Easy, Medium, Hard** difficulty options.
- **Replay** using the selected difficulty.
- **Exit** to close the game.
- A proper reset of the score, distance, obstacles, spawn timer, and speed.
- Preserve the collision fix and the final-score display from Prompts 1 and 2.

One important detail: the difficulty should control both the starting speed and how quickly the game gets faster, while keeping the maximum speed limit.

Step 1 — Add difficulty settings and reset logic

In `game/game_engine.py`, inside `GameEngine.__init__`, replace your existing speed and spawn configuration with this:

 Python



 Run

```
# Difficulty presets: starting speed, acceleration, spawn interval
self.difficulties = {
    "Easy": (4.5, 0.0015, 85),
    "Medium": (6.0, 0.003, 70),
    "Hard": (8.0, 0.004, 55),
}

self.max_speed = 12
self.selected_difficulty = "Medium"
self.speed = 6.0
```

Step 7 — Test everything

- ☐ Start the game and confirm jumping works.
- ☐ Collide with an obstacle and confirm the final score appears.
- ☐ Select Easy, Medium, and Hard and verify each has different starting speed and obstacle spawn frequency.
- ☐ Click Replay and confirm the score, obstacles, distance, and speed reset.
- ☐ Confirm the selected difficulty stays selected after replay.
- ☐ Check that fast obstacles still trigger collisions.
- ☐ Click Exit and confirm the game closes.

Step 8 — Commit Prompt 3

 Bash



```
git add game/game_engine.py main.py
git commit -m "Add replay and difficulty selection"
git push
```

Prompt 4 — Add sound effects

Add sound effects for jumping, scoring when passing an obstacle, and Game Over. Load the sounds from the project's assets folder and preserve the existing gameplay features.

We'll add three sounds:

- **Jump:** plays when the player jumps.
- **Score:** plays when the player successfully passes an obstacle.
- **Game Over:** plays once when a collision ends the run.

We'll load the sound files from the project's `assets` folder and preserve collision detection, difficulty selection, replay, and the final-score screen.

Run the game from your project folder:

 Bash



```
python main.py
```

Testing checklist

0 of 5

- ☐ **Jump sound plays when jumping**
Test Space, Up, and W.
- ☐ **Score sound plays when passing an obstacle**
It should play once per obstacle.
- ☐ **Game Over sound plays after collision**
It should play once per run.
- ☐ **Replay and difficulty selection still work**
Try Easy, Medium, and Hard.
- ☐ **Game runs even if audio is unavailable**
Check that missing audio doesn't crash the game.

Step 7 — Commit Prompt 4

Once everything works, run:

 Bash



```
git add game/game_engine.py sounds/  
git commit -m "Add jump score and game over sound effects"  
git push
```

Include the sound files in the commit if you added them. If the files were already tracked, Git will only record the changes.